

Android 第八章第一节教学讲义（高级控件与 Fragment 基础）

各位同学，今天咱们进入 Android 高级控件的学习！之前学的基础控件能满足简单界面需求，而高级控件（如下拉列表、列表控件、翻页控件）能帮我们实现更复杂的交互，比如商品列表、分页加载等。还会学习**Fragment（碎片）**——一种可以嵌入 Activity 的"迷你页面"，让界面更灵活。内容很重要，咱们用"原理 + 代码"的方式，保证小白也能轻松掌握！

一、下拉控件：Spinner（下拉选择框）

Spinner 就像一个"下拉菜单"，点击后弹出选项列表，适合从固定选项中选择（比如选择城市、性别）。

1. 基础用法：静态数据

步骤 1：布局中添加 Spinner

```
<Spinner
    android:id="@+id/spinner_city"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:prompt="请选择城市" /> <!-- 下拉框标题（仅弹窗显示） -->
```

步骤 2：准备数据源（数组）

在res/values/strings.xml中定义选项：

```
<string-array name="city_array">
    <item>北京</item>
    <item>上海</item>
    <item>广州</item>
    <item>深圳</item>
</string-array>
```

步骤 3：设置适配器（绑定数据）

适配器是 "数据源" 和 "控件" 之间的桥梁，Spinner 常用 `ArrayAdapter`：

```
Spinner spinner = findViewById(R.id.spinner_city);
// 1. 创建数组适配器（上下文、选项样式、数据源）
ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource(
    this,
    R.array.city_array, // 数据源（strings.xml中的数组）
    android.R.layout.simple_spinner_item // 选项样式（系统自带）
);
// 2. 设置下拉列表的样式
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
// 3. 绑定适配器到Spinner
spinner.setAdapter(adapter);
```

步骤 4：监听选择事件

```
spinner.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        // 选中项的文本（position是索引，从0开始）
        String selectedCity = parent.getItemAtPosition(position).toString();
        Toast.makeText(MainActivity.this, "选中了：" + selectedCity, Toast.LENGTH_SHORT).show
    };
}
@Override
public void onNothingSelected(AdapterView<?> parent) {
    // 没有选中时调用（很少用到）
}
});
```

二、列表类控件：RecyclerView（万能列表）

列表控件用于展示大量同类型数据（如商品列表、消息列表）。目前最推荐用**RecyclerView**，它比传统的 **ListView** 更灵活、性能更好，是 Android 开发的 "列表首选"。

1. 基本用法：展示简单文本列表

步骤 1：添加依赖（如果没有）

RecyclerView 在 AndroidX 库中，确保 `app/build.gradle` 有依赖：

```
implementation 'androidx.recyclerview:recyclerview:1.3.2'
```

步骤 2：布局中添加 RecyclerView

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/recycler_view"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent" />
```

步骤 3：创建列表项布局（每个 item 的样式）

在 `res/layout/item_text.xml` 中定义：

```
<TextView  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:id="@+id/tv_item"  
    android:layout_width="match_parent"  
    android:layout_height="50dp"  
    android:gravity="center"  
    android:textSize="16sp" />
```

步骤 4：创建适配器（核心）

适配器负责将数据绑定到列表项，RecyclerView 必须自定义适配器：

```
public class TextAdapter extends RecyclerView.Adapter<TextAdapter.ViewHolder> {  
    private List<String> mData; // 数据源  
    // 构造方法：接收数据源
```

```

public TextAdapter(List<String> data) {
    mData = data;
}
// 1. 创建ViewHolder（加载列表项布局）
@Override
public ViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
    View view = LayoutInflater.from(parent.getContext())
        .inflate(R.layout.item_text, parent, false);
    return new ViewHolder(view);
}
// 2. 绑定数据（将数据设置到列表项）
@Override
public void onBindViewHolder(ViewHolder holder, int position) {
    String text = mData.get(position);
    holder.tvItem.setText(text); // 给TextView设置文本
}
// 3. 返回数据总数
@Override
public int getItemCount() {
    return mData.size();
}
// ViewHolder：缓存列表项控件（避免重复 findViewById）
public static class ViewHolder extends RecyclerView.ViewHolder {
    TextView tvItem;
    public ViewHolder(View itemView) {
        super(itemView);
        tvItem = itemView.findViewById(R.id.tv_item);
    }
}
}

```

步骤 5：在 Activity 中初始化 RecyclerView

```

RecyclerView recyclerView = findViewById(R.id.recycler_view);
// 1. 设置布局管理器（决定列表的排列方式：线性、网格、瀑布流）
recyclerView.setLayoutManager(new LinearLayoutManager(this)); // 线性布局（垂直列表）
// 2. 准备数据源
List<String> data = new ArrayList<>();
for (int i = 0; i < 20; i++) {

```

```
data.add("列表项 " + (i + 1));  
}  
// 3. 设置适配器  
TextAdapter adapter = new TextAdapter(data);  
recyclerView.setAdapter(adapter);
```

2. 高级适配器：BaseAdapter（兼容 ListView）

如果需要兼容旧版本或使用 ListView，可以用BaseAdapter。以 ListView 为例：

步骤 1：布局添加 ListView

```
<ListView  
    android:id="@+id/list_view"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent" />
```

步骤 2：自定义 BaseAdapter

```
public class MyBaseAdapter extends BaseAdapter {  
    private List<String> mData;  
    private LayoutInflater mInflater;  
    public MyBaseAdapter(Context context, List<String> data) {  
        mData = data;  
        mInflater = LayoutInflater.from(context); // 用于加载布局  
    }  
    // 数据总数  
    @Override  
    public int getCount() {  
        return mData.size();  
    }  
    // 获取指定位置的数据  
    @Override  
    public Object getItem(int position) {  
        return mData.get(position);  
    }  
    // 获取item的ID（通常返回position）  
    @Override
```

```

public long getItemId(int position) {
    return position;
}
// 关键：返回列表项视图
@Override
public View getView(int position, View convertView, ViewGroup parent) {
    ViewHolder holder;
    // 3. convertView复用：避免重复创建视图（性能优化）
    if (convertView == null) {
        // 第一次加载：创建视图并缓存控件
        convertView = LayoutInflater.inflate(R.layout.item_text, parent, false);
        holder = new ViewHolder();
        holder.tvItem = convertView.findViewById(R.id.tv_item);
        convertView.setTag(holder); // 缓存ViewHolder
    } else {
        // 复用已有视图：直接从缓存取ViewHolder
        holder = (ViewHolder) convertView.getTag();
    }
    // 绑定数据
    holder.tvItem.setText(mData.get(position));
    return convertView;
}
// ViewHolder：缓存控件
static class ViewHolder {
    TextView tvItem;
}
}

```

步骤 3：在 Activity 中使用

```

ListView listView = findViewById(R.id.list_view);
List<String> data = new ArrayList<>();
for (int i = 0; i < 20; i++) {
    data.add("BaseAdapter 列表项 " + (i + 1));
}
listView.setAdapter(new MyBaseAdapter(this, data));

```

3. convertView 复用：列表性能优化的关键

为什么要复用convertView?

- 列表项很多时（如 100 项），屏幕一次只能显示 10 项；
- 如果每滑动一次就创建一个新视图，会浪费内存和 CPU 资源；
- 复用机制：只创建 "一屏幕 + 1" 个视图，滑动时回收不可见的视图，重新绑定数据。

效果：滑动列表更流畅，尤其数据量大时差距明显（上面的 BaseAdapter 已实现复用）。

三、下拉列表：SimpleAdapter（复杂列表项）

SimpleAdapter可以展示包含图片、文字的复杂列表项（如微信联系人列表：头像 + 名字）。

案例：展示带图片的列表

步骤 1：列表项布局（item_contact.xml）

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="60dp"
    android:gravity="center_vertical"
    android:padding="10dp">
    <ImageView
        android:id="@+id/iv_avatar"
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:src="@mipmap/ic_launcher" /> <!-- 头像 -->
    <TextView
        android:id="@+id/tv_name"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="10dp"
        android:textSize="16sp" /> <!-- 姓名 -->
</LinearLayout>
```

步骤 2：在 Activity 中使用 SimpleAdapter

```
ListView listView = findViewById(R.id.list_view);
// 1. 准备数据源 (List<Map>, 每个Map对应一个item的键值对)
List<Map<String, Object>> data = new ArrayList<>();
for (int i = 0; i < 10; i++) {
    Map<String, Object> item = new HashMap<>();
    item.put("avatar", R.mipmap.ic_launcher); // 头像资源ID
    item.put("name", "联系人 " + (i + 1)); // 姓名
    data.add(item);
}
// 2. 创建SimpleAdapter
SimpleAdapter adapter = new SimpleAdapter(
    this,
    data, // 数据源
    R.layout.item_contact, // 列表项布局
    new String[]{"avatar", "name"}, // Map中的key
    new int[]{R.id.iv_avatar, R.id.tv_name} // 对应布局中的控件ID
);
// 3. 设置适配器
listView.setAdapter(adapter);
```

四、翻页类控件：ViewPager（滑动切换页面）

ViewPager 用于实现左右滑动切换页面（如引导页、轮播图、多标签页），需要配合ViewPager2（最新版）或传统ViewPager。这里以ViewPager2为例（更推荐）。

1. 基础用法：滑动切换 3 个页面

步骤 1：布局添加 ViewPager2

```
<androidx.viewpager2.widget.ViewPager2
    android:id="@+id/view_pager2"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

步骤 2：创建 3 个页面布局（如 page1.xml、page2.xml、page3.xml）

以page1.xml为例：

```
<TextView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:background="#FF9900"
    android:text="页面 1"
    android:textSize="24sp" />
```

步骤 3：创建 ViewPager2 适配器

```
public class ViewPager2Adapter extends RecyclerView.Adapter<ViewPager2Adapter.
ViewHolder> {
    private List<Integer> mLayouts; // 存储页面布局ID
    public ViewPager2Adapter(List<Integer> layouts) {
        mLayouts = layouts;
    }
    @Override
    public ViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(mLayouts.get(viewType), parent, false);
        return new ViewHolder(view);
    }
    @Override
    public void onBindViewHolder(ViewHolder holder, int position) {
        // 无需绑定数据，直接显示布局
    }
    @Override
    public int getItemCount() {
        return mLayouts.size();
    }
    @Override
```

```
public int getItemViewType(int position) {
    return position; // 每个位置对应不同布局
}
static class ViewHolder extends RecyclerView.ViewHolder {
    public ViewHolder(View itemView) {
        super(itemView);
    }
}
}
```

步骤 4：在 Activity 中初始化

```
ViewPager2 viewPager2 = findViewById(R.id.view_pager2);
// 准备页面布局列表
List<Integer> layouts = new ArrayList<>();
layouts.add(R.layout.page1);
layouts.add(R.layout.page2);
layouts.add(R.layout.page3);
// 设置适配器
viewPager2.setAdapter(new ViewPager2Adapter(layouts));
```

五、Fragment：可复用的 "迷你页面"

Fragment（碎片）是一种可以嵌入 Activity 的 UI 组件，就像 "迷你 Activity"，有自己的布局 and 生命周期。适合在大屏幕设备（如平板）上实现多区域界面，也能让手机 APP 的界面更灵活。

1. 创建 Fragment 步骤

步骤 1：创建 Fragment 类

```
public class MyFragment extends Fragment {
    // 1. 加载布局
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
```

```

        @Nullable ViewGroup container,
        @Nullable Bundle savedInstanceState) {
    // 加载fragment布局
    return inflater.inflate(R.layout.fragment_my, container, false);
}
// 2. 视图创建后初始化（类似Activity的onCreate）
@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);
    // 在这里可以 findViewById 并设置事件
    TextView tv = view.findViewById(R.id.tv_fragment);
    tv.setText("这是一个Fragment");
}
}

```

步骤 2：创建 Fragment 布局（fragment_my.xml）

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:background="#E0E0E0">
    <TextView
        android:id="@+id/tv_fragment"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20sp" />
</LinearLayout>

```

步骤 3：在 Activity 中添加 Fragment

有两种方式：布局中静态添加 或 代码中动态添加。推荐**动态添加**（更灵活）。

动态添加代码：

```

public class FragmentActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_fragment);
    }
}

```

```
// 获取Fragment管理器
FragmentManager fragmentManager = getSupportFragmentManager();
// 开启事务
FragmentManager transaction = fragmentManager.beginTransaction();
// 替换布局中的容器为MyFragment (R.id.fragment_container是Activity布局中的一个-
FrameLayout)
transaction.replace(R.id.fragment_container, new MyFragment());
// 提交事务
transaction.commit();
}
}
```

Activity 布局 (activity_fragment.xml) :

```
<FrameLayout
    android:id="@+id/fragment_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

总结

这节课咱们学了 Android 高级控件的核心知识：

1. **下拉控件 Spinner**：用 `ArrayAdapter` 绑定静态数据，通过监听器获取选中项；
2. **列表类控件 RecyclerView**：万能列表，需自定义适配器，设置布局管理器（线性 / 网格）；
3. **BaseAdapter 与 convertView 复用**：适配 `ListView`，复用视图优化性能，`ViewHolder` 缓存控件；
4. **SimpleAdapter**：快速实现带图片、文字的复杂列表项，适合简单场景；
5. **翻页控件 ViewPager2**：左右滑动切换页面，适配器类似 `RecyclerView`；
6. **Fragment**：可复用的迷你页面，通过 `FragmentManager` 动态添加到 `Activity`。

这些控件是构建复杂 APP 界面的基础，尤其是 `RecyclerView` 和 `Fragment`，在实际开发中几乎天天用到。课后建议做一个 "新闻列表 APP"：用 `RecyclerView` 展示新闻标题列表，点击 item 跳转到包含 `Fragment` 的详情页，巩固今天的知识！有问题随时问～

（注：文档部分内容可能由 AI 生成）